7

Advanced Data Filtering

In this chapter, you'll learn how to combine WHERE clauses to create powerful and sophisticated search conditions. You'll also learn how to use the NOT and IN operators.

Combining WHERE Clauses

All the WHERE clauses introduced in Chapter 6, "Filtering Data," filter data using a single criterion. For a greater degree of filter control, MySQL allows you to specify multiple WHERE clauses. These clauses may be used in two ways: as AND clauses or as OR clauses.

Using the AND Operator

To filter by more than one column, you use the AND operator to append conditions to your WHERE clause. The following code demonstrates this:

► Input

```
SELECT prod_id, prod_price, prod_name
FROM products
WHERE vend_id = 1003 AND prod_price <= 10;
```

► Analysis

This SQL statement retrieves the product name and price for every product made by vendor 1003 as long as the price is 10 or less. The WHERE clause in this SELECT statement is made up of two conditions, and the keyword AND is used to join them. AND instructs MySQL to return only rows that meet all the conditions specified. If a product is made by vendor 1003 but costs more than 10, it is not retrieved. Similarly, products that cost less than 10 that are made by a vendor other than the one specified are not retrieved. The output generated by this SQL statement is as follows:

► Output

```
+---------+------------+----------------+
| prod_id | prod_price | prod_name      |
+---------+------------+----------------+
| FB      |      10.00 | Bird seed      |
| FC      |       2.50 | Carrots        |
| SLING   |       4.49 | Sling          |
```

```
| TNT1    |       2.50 | TNT (1 stick)  |
| TNT2    |      10.00 | TNT (5 sticks) |
+---------+------------+----------------+
```

New Term

AND A keyword used in a WHERE clause to specify that only rows matching all the specified conditions should be retrieved.

This example contains a single AND clause and is thus made up of two filter conditions. Additional filter conditions could be used as well, each separated by an AND keyword.

Using the OR Operator

The OR operator is exactly the opposite of AND. The OR operator instructs MySQL to retrieve rows that match either condition.

Look at the following SELECT statement:

► Input

```
SELECT prod_name, prod_price
FROM products
WHERE vend_id = 1002 OR vend_id = 1003;
```

► Analysis

This SQL statement retrieves the product name and price for any products made by either of the two specified vendors. The OR operator tells MySQL to match either condition—not both. If an AND operator were used here, no data would be returned. (It would create a WHERE clause that can never be matched.) The output generated by this SQL statement is as follows:

► Output

```
+---------------+------------+
| prod_name     | prod_price |
+---------------+------------+
| Detonator     |      13.00 |
| Bird seed     |      10.00 |
| Carrots       |       2.50 |
| Fuses         |       3.42 |
| Oil can       |       8.99 |
| Safe          |      50.00 |
| Sling         |       4.49 |
| TNT (1 stick) |       2.50 |
| TNT (5 sticks) |      10.00 |
+---------------+------------+
```

New Term

OR A keyword used in a WHERE clause to specify that any rows matching either of the specified conditions should be retrieved.

Understanding the Order of Evaluation

WHERE clauses can contain any number of AND and OR operators. By combining the two operators, you can perform sophisticated and complex filtering.

But combining AND and OR operators presents an interesting problem. For example, say that you need a list of all products that cost 10 or more and that were made by vendors 1002 and 1003. The following SELECT statement uses a combination of AND and OR operators to build a WHERE clause for this search:

► Input

```
SELECT prod_name, prod_price
FROM products
WHERE vend_id = 1002
   OR vend_id = 1003
   AND prod_price >= 10;
```

► Output

```
+----------------+------------+
| prod_name      | prod_price |
+----------------+------------+
| Detonator      |      13.00 |
| Bird seed      |      10.00 |
| Fuses          |       3.42 |
| Oil can        |       8.99 |
| Safe           |      50.00 |
| TNT (5 sticks) |      10.00 |
+----------------+------------+
```

► Analysis

Look at these results. Two of the rows returned have prices less than 10—so, obviously, the rows were not filtered as intended. Why did this happen? Because of the order of evaluation. SQL (like most other languages) processes AND operators before OR operators. When SQL sees the preceding WHERE clause, it reads "products made by vendor 1002 regardless of price and any products costing 10 or more made by vendor 1003." In other words, because AND ranks higher in the order of evaluation, the wrong operators are joined together here.

The solution to this problem is to use parentheses to explicitly group related operators. Take a look at the following SELECT statement and its output:

► Input

```
SELECT prod_name, prod_price
FROM products
```

```
WHERE (vend_id = 1002 OR vend_id = 1003)
   AND prod_price >= 10;
```

Output

```
+---------------+------------+
| prod_name     | prod_price |
+---------------+------------+
| Detonator     |      13.00 |
| Bird seed     |      10.00 |
| Safe          |      50.00 |
| TNT (5 sticks) |      10.00 |
+---------------+------------+
```

Analysis

The only difference between this `SELECT` statement and the earlier one is that, in this statement, the first two `WHERE` clause conditions are enclosed within parentheses. Because parentheses have a higher order of evaluation than either `AND` or `OR` operators, MySQL first filters the `OR` condition within those parentheses. The SQL statement then looks for any products made by either vendor 1002 or vendor 1003 that cost 10 or greater, which is exactly what we want.

Tip

Using Parentheses in `WHERE` Clauses Whenever you write `WHERE` clauses that use both `AND` and `OR` operators, use parentheses to explicitly group the operators. Don't ever rely on the default evaluation order, even if it is exactly what you want. There is no downside to using parentheses, and you are always better off eliminating any ambiguity.

Using the `IN` Operator

Parentheses have another very different use in `WHERE` clauses. The `IN` operator is used to specify a range of conditions, any of which can be matched. `IN` takes a comma-delimited list of valid values, all enclosed within parentheses. The following example demonstrates this:

Input

```
SELECT prod_name, prod_price
FROM products
WHERE vend_id IN (1002,1003)
ORDER BY prod_name;
```

Output

```
+---------------+------------+
| prod_name     | prod_price |
+---------------+------------+
| Bird seed     |      10.00 |
```

```
| Carrots        |       2.50 |
| Detonator      |      13.00 |
| Fuses          |       3.42 |
| Oil can        |       8.99 |
| Safe           |      50.00 |
| Sling          |       4.49 |
| TNT (1 stick)  |       2.50 |
| TNT (5 sticks) |      10.00 |
+----------------+------------+
```

► Analysis

The SELECT statement retrieves all products made by vendors 1002 and 1003. The IN operator is followed by a comma-delimited list of valid values, and the entire list must be enclosed within parentheses.

You might be thinking that the IN operator accomplishes the same goal as OR—and that is correct. The following SQL statement accomplishes exactly the same thing as the previous example:

► Input

```
SELECT prod_name, prod_price
FROM products
WHERE vend_id  = 1002 OR vend_id = 1003
ORDER BY prod_name;
```

► Output

```
+----------------+------------+
| prod_name      | prod_price |
+----------------+------------+
| Bird seed      |      10.00 |
| Carrots        |       2.50 |
| Detonator      |      13.00 |
| Fuses          |       3.42 |
| Oil can        |       8.99 |
| Safe           |      50.00 |
| Sling          |       4.49 |
| TNT (1 stick)  |       2.50 |
| TNT (5 sticks) |      10.00 |
+----------------+------------+
```

There are several advantages to using the IN operator:

- When you are working with long lists of valid options, the IN operator syntax is far cleaner and easier to read.
- The order of evaluation is easier to manage when IN is used (as fewer operators are used).
- IN operators almost always execute more quickly than lists of OR operators.
- The biggest advantage of IN is that the IN operator can contain another SELECT statement, enabling you to build highly dynamic WHERE clauses. You'll look at this in detail in Chapter 14, "Working with Subqueries."

New Term

IN A keyword used in a WHERE clause to specify a list of values to be matched using an OR comparison.

Using the NOT Operator

The WHERE clause's NOT operator has one function and one function only: It negates whatever condition comes next.

New Term

NOT A keyword used in a WHERE clause to negate a condition.

The following example demonstrates the use of NOT. To list the products made by all vendors except vendor DLL01, you can use the following:

Input

```
SELECT prod_name
FROM Products
WHERE NOT vend_id = 'DLL01'
ORDER BY prod_name;
```

Output

```
+----------------+
| prod_name      |
+----------------+
| .5 ton anvil   |
| 1 ton anvil    |
| 2 ton anvil    |
| Bird seed      |
| Carrots        |
| Detonator      |
| Fuses          |
| JetPack 1000   |
| JetPack 2000   |
| Oil can        |
| Safe           |
| Sling          |
| TNT (1 stick)  |
| TNT (5 sticks) |
+----------------+
```

Analysis

The NOT here negates the condition that follows it. So instead of matching vend_id to DLL01, MySQL matches vend_id to anything that is not DLL01.

This example could also be written using != instead of NOT (as you saw in Chapter 6):

Input

```
SELECT prod_name
FROM Products
WHERE vend_id  != 'DLL01'
ORDER BY prod_name;
```

Actually, it could also be written as:

Input

```
SELECT prod_name
FROM Products
WHERE vend_id  <> 'DLL01'
ORDER BY prod_name;
```

Analysis

Why use NOT? Well, for simple WHERE clauses such as the ones shown here, there really is no advantage to using NOT.

But NOT is extremely useful in more complex clauses. For example, using NOT in conjunction with an IN operator makes it simple to find all rows that do not match a list of criteria. The following example demonstrates this. To list the products made by all vendors except vendors 1002 and 1003, you can use the following:

Input

```
SELECT prod_name, prod_price
FROM products
WHERE vend_id NOT IN (1002,1003)
ORDER BY prod_name;
```

Output

```
+--------------+------------+
| prod_name    | prod_price |
+--------------+------------+
| .5 ton anvil |       5.99 |
| 1 ton anvil  |       9.99 |
| 2 ton anvil  |      14.99 |
| JetPack 1000 |      35.00 |
| JetPack 2000 |      55.00 |
+--------------+------------+
```

Analysis

The NOT here negates the condition that follows it. So instead of matching vend_id to 1002 or 1003, MySQL matches vend_id to anything that is not 1002 or 1003.

So why use NOT? Well, for simple WHERE clauses, there really is no advantage to using NOT. NOT is useful in more complex clauses. For example, using NOT in conjunction with an IN operator makes it easy to find all rows that do not match a list of criteria.

Tip

There Is Often More Than One Solution As you've seen here, there is frequently more than one way to write a SQL statement. When you're working with large sets of data, there may be performance differences, such as one statement being faster than another. But for smaller data sets, the syntax used is usually a matter of personal preference.

Note

`NOT` in MySQL MySQL supports the use of `NOT` to negate `IN`, `BETWEEN`, and `EXISTS` clauses. This is quite different from most other DBMSs, which allow `NOT` to be used to negate any conditions.

Summary

This chapter picked up where the previous chapter left off and taught you how to combine `WHERE` clauses with the `AND` and `OR` operators. You also learned how to explicitly manage the order of evaluation and how to use the `IN` and `NOT` operators.

Challenges

1. Write a SQL statement to retrieve the vendor name (`vend_name`) from the `Vendors` table and return only vendors in California. (This requires filtering by both country [`USA`] and state [`CA`]; after all, there could be a California outside of the United States.) Here's a hint: The filter requires matching strings.
2. Write a SQL statement to find all orders where at least 100 of items BR01, BR02, or BR03 were ordered. You'll want to return the order number (`order_num`), product ID (`prod_id`), and quantity for the `OrderItems` table and filter by both the product ID and quantity. Here's a hint: Depending on how you write your filter, you may need to pay special attention to the order of evaluation.
3. Now let's revisit a challenge from the previous lesson. Write a SQL statement that returns the product name (`prod_name`) and price (`prod_price`) from `Products` for all products priced between 3 and 6. Use the `AND` operator and sort the results by price.
4. What is wrong with the following SQL statement? Try to figure it out without running it.

```
SELECT vend_name
FROM Vendors
ORDER BY vend_name
WHERE vend_country = 'USA' AND vend_state = 'CA';
```